M. Frisbie, *Building Browser Extensions*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_9

9. Content Scripts

Matt Frisbie[1]

(1) California, CA, USA

Contents scripts are one of the most powerful tools available in browser extensions. They allow you to inject JavaScript and CSS into any web page and modify it with almost no restrictions. The injected content can be as simple as a few visual tweaks, or as complicated as entire single page application frameworks. Extensions can also use them for nonuser interface reasons: content scripts can read and modify the page DOM, as well as send network requests as the authenticated user.

Introduction to Content Scripts

The most common way that content scripts are used is by declarative injection via the manifest. The manifest specifies the files that should be injected, and the domains in which they should be injected, and the browser will inject them into the page. Injected JavaScript is analogous to dynamically inserting a `<script>` tag into the page, and injected CSS is analogous to dynamically inserting a `<link>` tag into the page. The following example demonstrates an extension that will inject JS and CSS into all web pages:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "content_scripts": [
```

```
    {
      "matches": ["<all_urls>"],
      "css": ["content-script.css"],
      "js": ["content-script.js"]
    }
  ],
  "permissions": []
}
```

Example 9-1a manifest.json

```
console.log(window.jQuery);
document.body.innerHTML = "Hello, world!";
```

Example 9-1b content-script.js

```
body {
    background-color: red !important;
}
```

Example 9-1c content-script.css

Injecting CSS

Injecting CSS content scripts into a host page is relatively straightforward. If the URL is a match, the script will be injected as though it were listed as an extra `<link>` element. Install the previous example extension and visit any website to see the CSS injection result (Figure **9-1**).

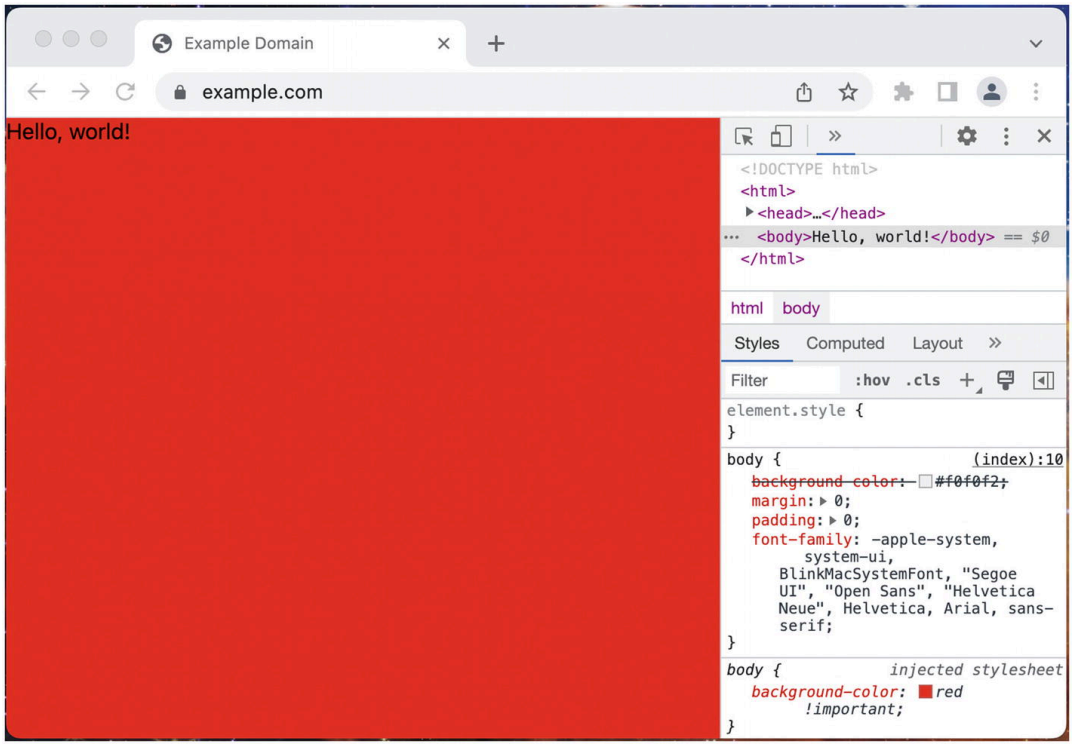

Figure 9-1 example.com with the content script's CSS applied

There are a few things to notice here:

- The content scripts are only projected into the page. You will not see actual `<script>` or `<link>` tags added.
- The injected CSS specificity is less than the page's CSS, so using `!important` is often necessary.

Content Script Isolation

Next, navigate to `jquery.com` (or any other website that loads the jQuery library) (Figure 9-2).

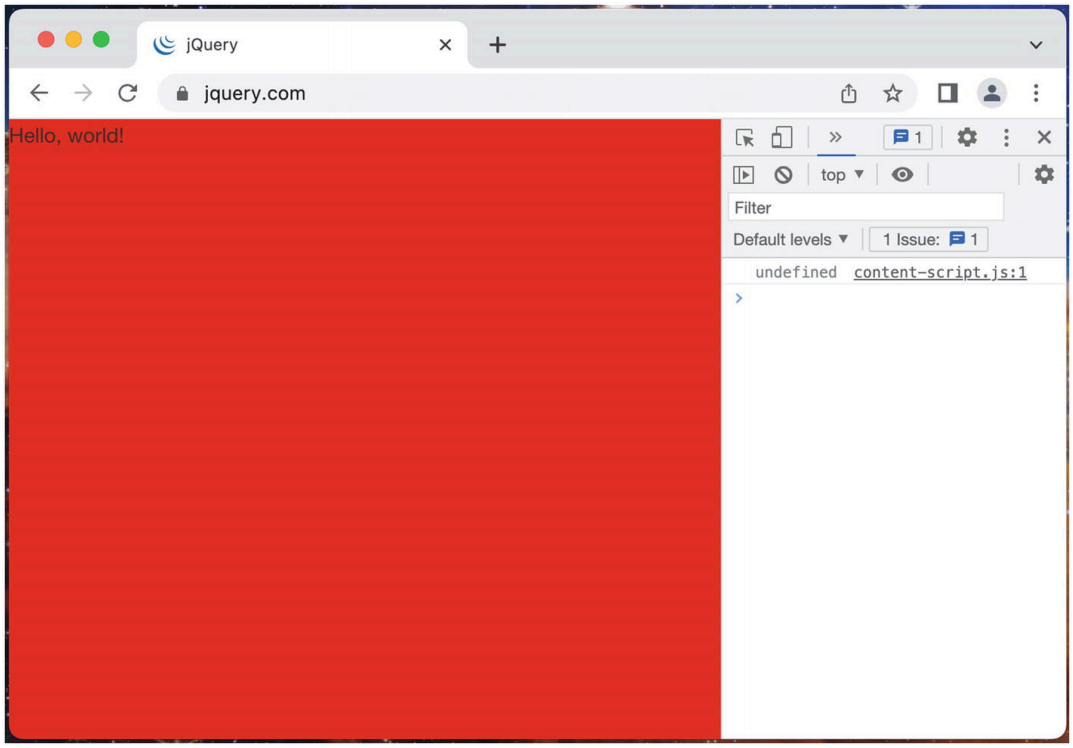

Figure 9-2 jquery.com with the content script's log output

When the jQuery JavaScript library is loaded on `jquery.com`, the `jQuery` property is defined on the `window` object, but the content script is unable to see this property. However, in the developer tools, `window.jQuery` will be defined. This is due to the content script's run-time isolation.

Content script JavaScript executes in a separate runtime in parallel to the host page. Neither runtime can access the other's variables, and each has its own event loop, global scope, and task queue. However, both can access APIs that interact with the same objects: for example, the DOM, `localStorage`, `sessionStorage`, `IndexedDB`, `cookieStore`, etc. Suppose the host page defines a variable in the global scope. The content script has no way of accessing its value. However, if the host page were to write that variable into `localStorage`, the content script's `localStorage` API would have access to that value!

Shared access to the DOM is particularly interesting. One immediately useful feature is the ability of a content script to add, modify, or remove DOM nodes from the host page. For example, if you wanted to wipe out all the CSS styling on all web pages, you could use the following example extension:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "css": [],
      "js": ["content-script.js"]
    }
  ],
  "permissions": []
}
```

Example 9-2a manifest.json

```
for (const el of document.querySelectorAll("style")) {
  el.parentElement.removeChild(el);
}
for (const el of document.querySelectorAll('link[rel="stylesheet"]')) {
  el.parentElement.removeChild(el);
}
for (const el of document.querySelectorAll("[style]")) {
  el.removeAttribute("style");
}
```

Example 9-2b content-script.js

Load this extension and navigate to any web page. You will notice that all the CSS styling will be completely stripped out.

> **Note** This will not affect CSS styling that is added by JavaScript after the content script runs. One trivial solution might be to wrap this script in a `setInterval()`, but then there may be performance considerations to take into account with all the extra `querySelectorAll` expressions being evaluated. This sort of page management is what makes content scripts so tricky to get right.

Page Automation

Although a content script cannot access event handlers, it can dispatch events on shared DOM nodes. Event handlers that the host page assigned in the host JavaScript context will be called by events dispatched from the content script JavaScript context. The following example demonstrates this by automating a search on `Wikipedia.org`.

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "content_scripts": [
    {
      "matches": ["https://*.wikipedia.org/*"],
      "css": [],
      "js": ["content-script.js"]
    }
  ],
  "permissions": []
}
```

Example 9-3a manifest.json

```
// Wait a few seconds so the user
// can see the query being entered
setTimeout(() => {
  document.querySelector("#searchInput").value = "javascript";
}, 2000);
setTimeout(() => {
  document.querySelector('button[type="submit"]').click();
}, 3000);
```

Example 9-3b content-script.js

After loading this extension, visit **`https://wikipedia.org`**. You will see the extension set a value in the input field and click the search button, and you will be taken to the page on JavaScript.

If you wanted to make the text entry more realistic, you could adjust the background script to the following:

content-script.js

```
const typedValue = "javascript";
const input = document.querySelector("#searchInput");
const form = document.querySelector("#search-form");
function typeOrSubmit(idx = 0) {
  const char = typedValue[idx];
  if (!char) {
    setTimeout(() => form.submit(), 500);
  } else {
    input.value = input.value + char;
    setTimeout(() => typeOrSubmit(++idx), 100);
  }
}
if (input && form) {
  setTimeout(() => {
    input.focus();
    typeOrSubmit();
  }, 2000);
}
```

Reload the extension and the Wikipedia page. The content script will progressively type out the search term. Also note here that the content script is submitting the form directly, not just clicking the search button.

This touches upon an important concept to keep in mind when writing content scripts: you are at the mercy of the host page. In the above example, we are using selectors defined by the host page to locate the elements we want to interact with. If the host page changes these selectors even a small amount, the content script will break. Furthermore, note that the content scripts are using built-in event dispatching methods like `click()`, `focus()`, and `submit()`. These are handy because they are succinct and partially allow you the ability to avoid dispatching events manually.

Sometimes, dispatching events manually is required. A content script can't see which elements have event handlers attached, but the developer can find this information out ahead of time using the `getEventListeners()` method. For example, on **https://wikipedia.org**, you can use this method to find all the event listeners on the "Read Wikipedia in your language" button (Figure 9-3).

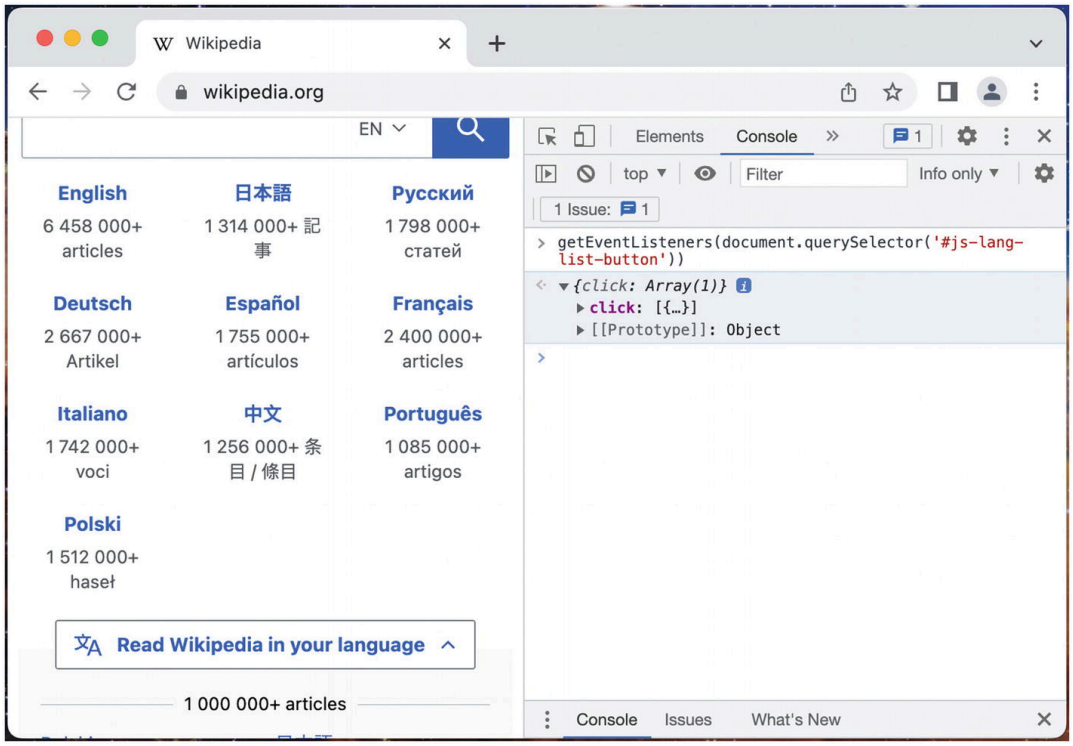

Figure 9-3 The result of getEventListeners()

We see here there is an event listener set by the host page. We can then check what this handler is doing by just dispatching a test click event in the developer console – and you should see it toggle the language menu. To automate the opening of this menu, we could certainly call `click()` in the content script, but for the sake of this example let's instead open the menu by manually dispatching a click event:

content-script.js

```
setTimeout(() => {
  const el = document.querySelector("#js-lang-list-button");
  // Scroll down to the button so we can see the click work
  el.scrollIntoView();
```

```
    el.dispatchEvent(new Event("click"));
  }, 2000);
```

Reload the extension and the Wikipedia page. You should see the content script scroll the page down and automatically open the language menu.

Logging and Errors

Console messages and errors produced by a content script will show up in the host page's developer console (Figure 9-4).

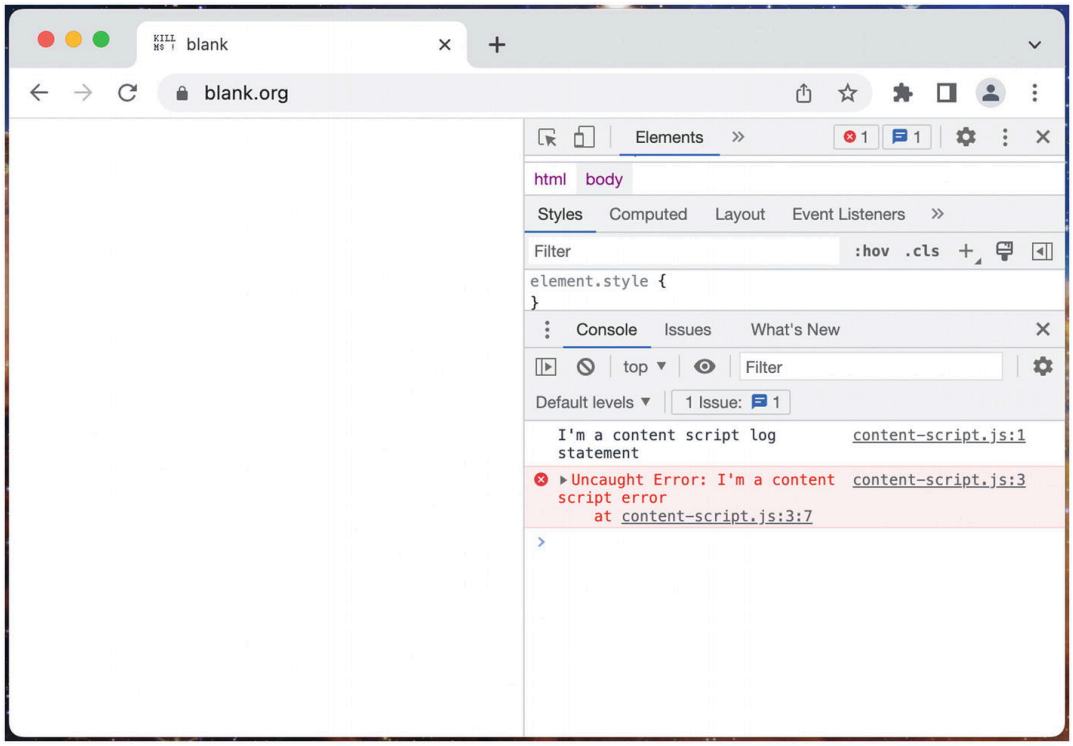

Figure 9-4 Console messages and errors from a content script

Because uncaught errors thrown in the content script are still part of the extension context, these error messages will also surface in the extension's error view (Figure 9-5).

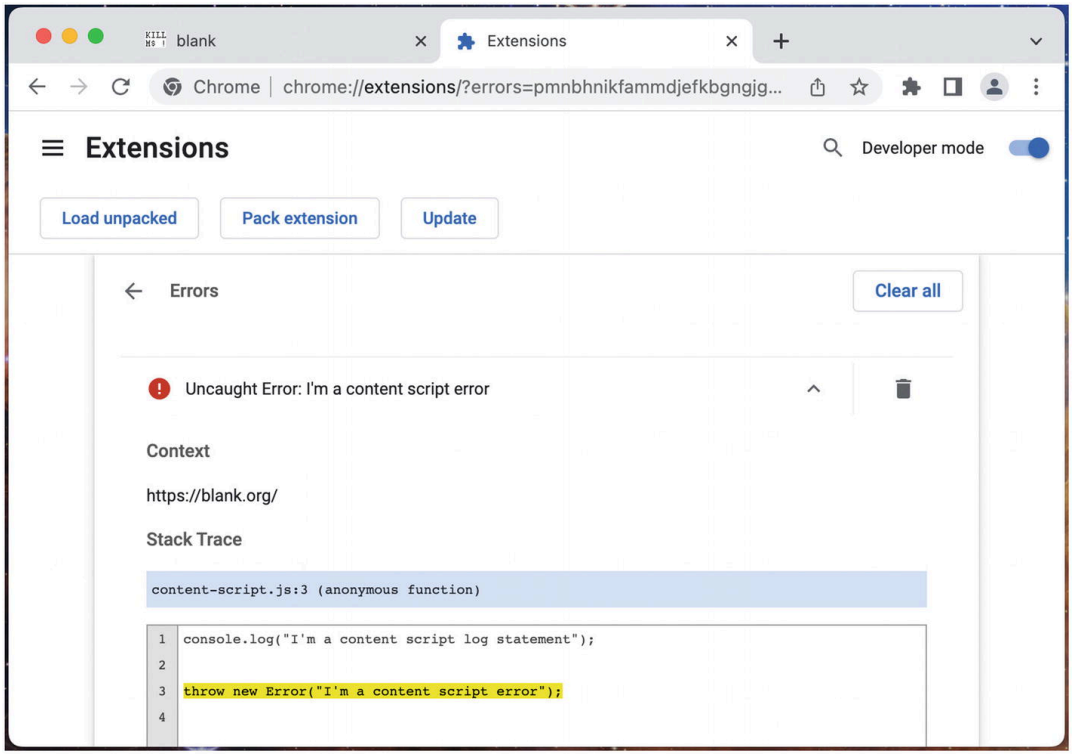

Figure 9-5 Errors visible in the extension error view

Extension API Access

Unlike popups and options pages, content scripts can only access a limited portion of the WebExtensions API:

- `chrome.i18n.*`
- `chrome.storage.*`
- `chrome.runtime.connect`
- `chrome.runtime.getManifest`
- `chrome.runtime.getURL`
- `chrome.runtime.id`
- `chrome.runtime.onConnect`
- `chrome.runtime.onMessage`
- `chrome.runtime.sendMessage`

If you require other parts of the API, you can delegate to the background service worker by sending a message to trigger a remote procedure call. The background service worker can use the full API and send back the result if needed.

One unusual aspect of content scripts is, although they can use `runtime.getURL()`, they are prevented from opening tabs of extension URLs. The tab will open, but the page will not render and instead show a browser error. Content scripts run in an untrusted environment, and therefore using `<a href>` or `window.open(chrome-extension://...)` to access extension pages would necessarily mean that the host pages could open those URLs too. As shown in the *Background Scripts* chapter, the solution to this is to send a message to the background directing it to open a new tab with the desired URL.

Modules and Code Splitting

As any experienced developer knows, modern JavaScript makes heavy use of ES6 modules and the `import` keyword. Because the top-level content script is not a module, and there is currently no way to define it as a module, it cannot have static imports. Fortunately, there are a handful of straightforward solutions.

Bundling

Most extension development is done using sophisticated build tools like Parcel, Webpack, or Plasmo, and these are usually configured to smush the entire content script code graph into a single file, thereby eliminating the need for using imports in the top-level content script. Traditional web applications have more of a need for lazy loading, since from a performance perspective loading data from a remote server is expensive. Since browser extensions are exclusively served from the extension file server on the local device, there is a negligible penalty for bundling the entire content script into a gigantic monolithic script.

Dynamic Imports

Content scripts may not be able to use static imports, but they are perfectly happy using dynamic imports and loading a secondary module that *can* use static imports. Doing so requires an additional network request to the extension file server, but this penalty is negligible and therefore acceptable.

Of course, any static or dynamic `import` in a content script will generate a network request for that module. Therefore, to allow these modules to be imported, they must be listed as under `web_accessible_resources`. This is demonstrated in the following example extension, which performs a dynamic import. Load the extension and inspect the console output:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "css": [],
      "js": ["content-script.js"]
    }
  ],
  "permissions": [],
  "web_accessible_resources": [
    {
      "resources": ["*.js"],
      "matches": ["<all_urls>"]
    }
  ]
}
```

Example 9-4a manifest.json

```
const url = chrome.runtime.getURL("foo.js");
import(url).then((fooModule) => {
  fooModule.bar();
});
```

Example 9-4b content-script.js

```
import apiTest from "./api-test.js";
export function bar() {
  console.log("Called bar!");
}
console.log("Loaded foo module!");
apiTest();
```

Example 9-4c foo.js

```
export default function () {
  // You will only see this method present if
  // the script has access to the WebExtensions API
  console.log("Can access API:", !!chrome.runtime.getURL);
}
```

Example 9-4d api-test.js

The result is shown in Figure 9-6.

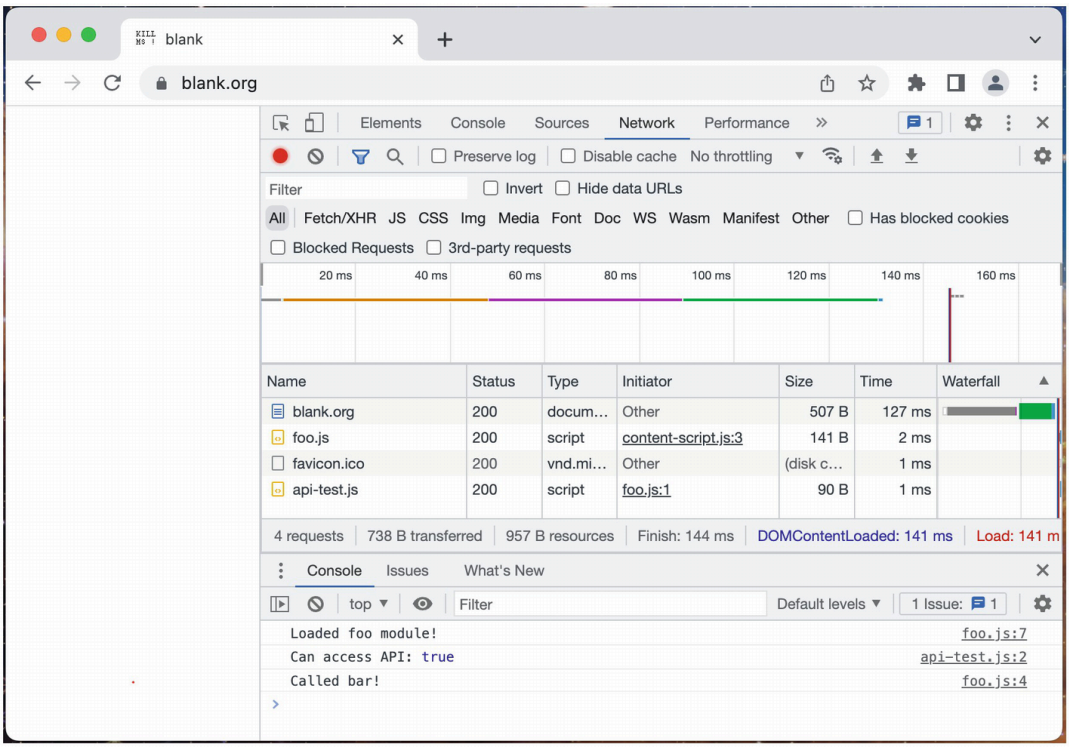

Figure 9-6 Dynamic module import network requests and console output

There are a few things to note here:

- As expected, both the static and dynamic `import` incurred network requests for the module files.
- The imported module retains access to the WebExtensions API and can import other modules.
- Because it's a dynamic import, you can call methods on the imported module from the returned module object.

Dynamic Script Tags

It is also possible to load modules in a similar fashion by dynamically creating a `<script type="module">` in the page. Change the previous example extension to match the following:

```
const url = chrome.runtime.getURL("foo.js");
const script = document.createElement("script");
script.setAttribute("type", "module");
script.setAttribute("src", url);
document.head.appendChild(script);
```

Example 9-5 content-script.js

Reload the extension and you will see the screen shown in Figure 9-7.

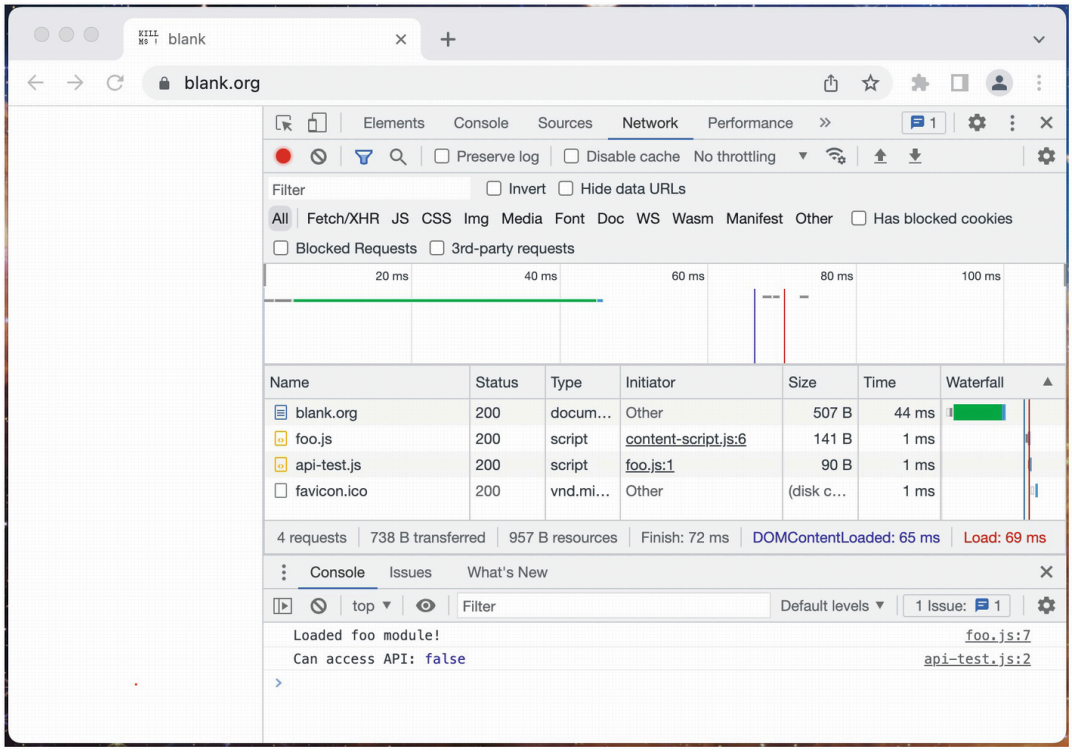

Figure 9-7 Dynamic script tag network requests and console output

There are a few things to note here:

- Dynamically created script tags lose access to the WebExtensions API.
- Because this isn't using the `import` keyword, we are unable to use specific exports from the module.

You may encounter some special situations where dynamic script tag creation is useful, but overall it is less useful than using dynamic imports.

Specialized Content Script Properties

Content scripts can be further customized to control when the script is injected, which URL paths it should or should not inject on, and whether or not it should inject in special URLs like `about:blank`. The following properties are available:

- `run_at`
- `match_about_blank`
- `match_origin_as_fallback`

- `exclude_matches`
- `include_globs`
- `exclude_globs`
- `all_frames`

Their behavior is detailed in the *Extension Manifests* chapter.

Programmatic Injection

The manifest is not the only way to inject content scripts. It is also possible to programmatically inject JavaScript and CSS into the page using the `chrome.scripting` API. Consider the following example that injects both JS and CSS after the toolbar icon is clicked:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "permissions": ["scripting", "activeTab"],
  "action": {}
}
```

Example 9-6a manifest.json

```
chrome.action.onClicked.addListener((tab) => {
  const target = {
    tabId: tab.id,
  };
  chrome.scripting.executeScript({
    target,
    func: () => {
      document.body.innerHTML = `Hello, world!`;
    },
  });
  chrome.scripting.insertCSS({
```

```
    target,
    css: `body { background-color: red !important; }`,
  });
});
```

Example 9-6b background.js

As shown here, the `chrome.scripting` API can be used to easily inject JavaScript and CSS into the page on an ad-hoc basis. The mechanism by which the function is passed into the page is somewhat unusual: it is effectively calling `func.toString()`, and then evaluating that string in the context of the page.

Instead of passing in functions and strings, it is also possible to provide file references. The following example behaves identically to the previous one:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "permissions": ["scripting", "activeTab"],
  "action": {}
}
```

Example 9-7a manifest.json

```
chrome.action.onClicked.addListener((tab) => {
  const target = {
    tabId: tab.id,
  };
  chrome.scripting.executeScript({
    target,
    files: ["content-script.js"],
  });
  chrome.scripting.insertCSS({
    target,
```

```
    files: ["content-script.css"],
  });
});
```

Example 9-7b background.js

```
document.body.innerHTML = `Hello, world!`;
```

Example 9-7c content-script.js

```
body {
  background-color: red !important;
}
```

Example 9-7d content-script.css

Importantly, no function closure is captured during serialization; any outside variables references are lost. Instead, variables can be serialized and curried into the function via an `args` property. The following example demonstrates this:

```
const outerVar = "foobar";
function wipeOutPage(bg) {
  // Record the typeof inside the content script
  const cs = typeof outerVar;
  document.body.innerHTML = `${bg} -> ${cs}`;
}
const css = `
body {
    background-color: red !important;
}`;
chrome.action.onClicked.addListener((tab) => {
  const target = {
    tabId: tab.id,
  };
  // Record the typeof inside the background
  const backgroundTypeof = typeof outerVar;
  chrome.scripting.executeScript({
    target,
    func: wipeOutPage,
    // This array of values will be curried
```

```
    // into `func` (similar to Array.apply)
    args: [backgroundTypeof]
  });
  chrome.scripting.insertCSS({
    target,
    css,
  });
});
```

Example 9-8 background.js

Reload this extension. You will see that the page content is `string -> undefined`. This indicates that the variable reference has been lost when the function is evaluated in the page.

> **Note** It was formerly possible to pass in a function string to `executeScript()`. In manifest v3 this is no longer possible, as this would enable arbitrary code execution

It is also possible to register and unregister declarative content scripts on the fly. This is analogous to updating the manifest `content_scripts` property. Since this is modifying the declarative injection list, the scripts will not be injected until the next pageload. The following example is an extension which toggles the declarative injection when the toolbar icon is clicked.

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "permissions": ["scripting", "activeTab"],
  "action": {}
}
```

Example 9-9a manifest.json

```
const id = "1";
chrome.action.onClicked.addListener(async () => {
```

```
  const activeScripts = await chrome.scripting.getRegisteredContentScripts();
  // Toggle the content script
  if (activeScripts.find((x) => x.id === id)) {
    chrome.scripting.unregisterContentScripts({
      ids: [id],
    });
    console.log("Unregistered content script");
  } else {
    chrome.scripting.registerContentScripts([
      {
        id,
        matches: ["<all_urls>"],
        js: ["content-script.js"],
        css: ["content-script.css"],
      },
    ]);
    console.log("Registered content script");
  }
});
```

Example 9-9b background.js

```
document.body.innerHTML = "Hello, world!";
```

Example 9-9c content-script.js

```
body {
    background-color: red !important;
}
```

Example 9-9d content-script.css

Summary

In this chapter, you learn about how content scripts allow you to exert almost total control over host web pages. Although they have limited access to the WebExtensions API, content scripts can coordinate with the background service worker to add profoundly powerful enhance-

ments to the user's browser experience. Finally, you were shown how content scripts can be dynamically added and removed from the page.

In the next chapter, you will learn how to build custom devtools pages into your browser's developer tools interface. You will also learn how to use the custom devtools APIs that are only accessible from an extension's devtools pages.